FPT SCHOOL OF BUSINESS & TECHNOLOGY

AI - MACHINE LEARNING

PhD. Truc Thi Kim, Nguyen

093 293 9981 info@fsb.edu.vn https://fsb.edu.vn

BINARY CLASSIFICATION

FSB

Binary Classification

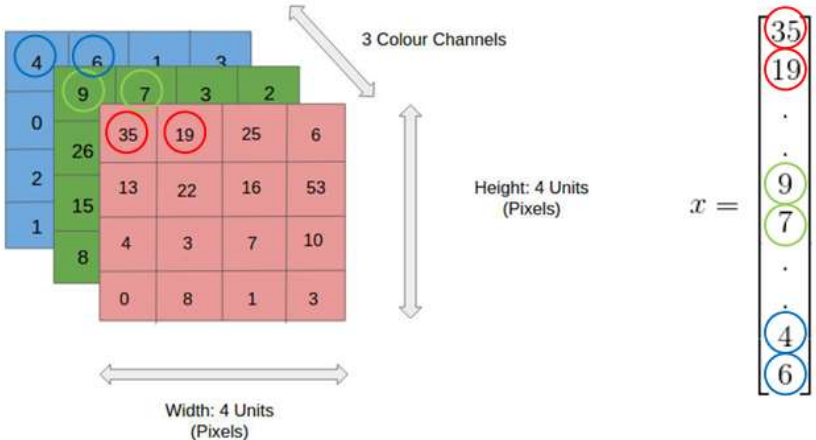
- Logistic regression is an algorithm for **binary classification**
- Example of a **binary classification** problem:
  - you have an **input (x)** of an image and
  - the **output (y)** is a label to recognize the image, 1 (cat) vs 0 (non cat)
- In binary classification our goal is to learn a classifier that can input an image represented by its feature vector X and predict whether the corresponding label y is 1 or 0, that is whether this is a cat image or a non cat image

1 (cat) vs **0** (non cat)

y

Binary Classification

- **Image representation in computer**: The computer stores 3 separate matrices corresponding to the red, green and blue color channels image
- If the input image is **64 by 64 pixel**, then you would have three 64 by 64 matrices corresponding to the red, green and blue pixel intensity values for your image.
- **64 by 64 image -** the **total dimension** of this vector **X** will be: n_x**=64*64*3=12288**



Binary Classification

Notation

- Single training example is represented by (x, y) , where $x \in \mathbb{R}^{n_x}$, label $y \in \{0, 1\}$
- Training set **m,** training examples: $\{ (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)}) \}$
- All **training examples**:
- **Output**:

$$X = \begin{bmatrix} \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ x^{(1)} & x^{(2)} & \ldots & x^{(m)} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \end{bmatrix} \begin{matrix} \uparrow \\ \\ n_x \\ \\ \downarrow \end{matrix}$$

$$\longleftarrow \quad m \quad \longrightarrow$$

$$y = \left[y^{(1)}, \quad y^{(2)}, \quad \ldots \quad , \quad y^{(m)} \right]$$

- $\boldsymbol{X}$ is $\boldsymbol{n_x}$ x $\boldsymbol{m}$ dimensional matrix: $x \in \mathbb{R}^{n_x x m}$
- Python command for finding the shape of X matrix is *X.shape = (n,m)*
- $\boldsymbol{y}$ is $\mathbf{1}$x $\boldsymbol{m}$ dimensional matrix: $y \in \mathbb{R}^{1xm}$
- Python command for finding the shape of y is *y.shape = (1,m)*

LOGISTIC REGRESSION

FSB

Logistic Regression

- Logistic regression is a **supervised learning** algorithm that we can use when the output labels are all either: **0** or **1**.
- This is, so called, **binary classification** problem.
- An input feature vector ***x*** may be corresponding to an image that we want to recognize as either a **cat picture (1)** or **not a cat picture (0)**.
- That is, we want an algorithm that can output the prediction ($\hat{y}$) which is an estimate of ***y***:

$$\hat{y} = P(y = 1|x)$$

$$x \in \mathbb{R}^{n_x}$$

$$\text{Parameters} : \omega \in R^{n_x}, b \in \mathbb{R}$$

Logistic Regression

- More formally, we want *y* to be the probability of the chance that *y* is equal to 1, given the input features *x*.
- In other words, if *x* is a picture, we want $\hat{y}$ to tell us what is the chance that this is a cat picture.
- An *x* is an *nx* dimensional vector.
- The parameters of logistic regression will be *w* which is also an *nx* dimensional vector together with *b* which is just a real number.
- Given an input x and the parameters *w* and *b*, how do we generate the output $\hat{y}$?
- One thing we could try, that **doesn't work**, would be to have: $\hat{y} = w^T x + b$

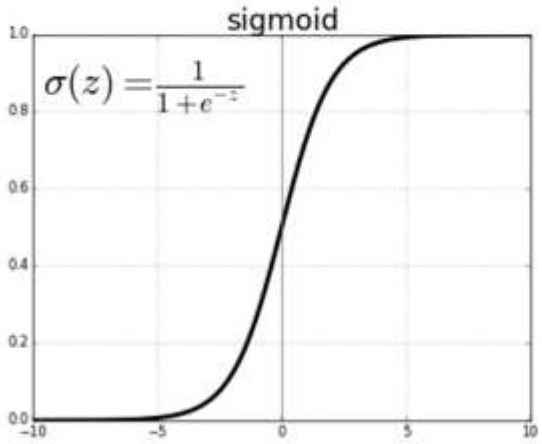
Logistic Regression

- $\hat{y} = w^T x + b$ is a **linear function** of the input *x* and in fact this is what we **use** if we were doing **linear regression**.
- However, **this isn't a very good algorithm for binary classification**, because we want $\hat{y}$ to be the chance that *y* is equal to 1, so $\hat{y}$ should be **between 0** and **1**.
- It is difficult to enforce, because $w^T x + b$ can be much bigger than 1 or can even be negative which doesn't make sense for a probability that we want to be between 0 and 1.
- So, in logistic regression our output is going to be the **Sigmoid function**.

Sigmoid Function of z

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- It goes smoothly from 0 up to 1



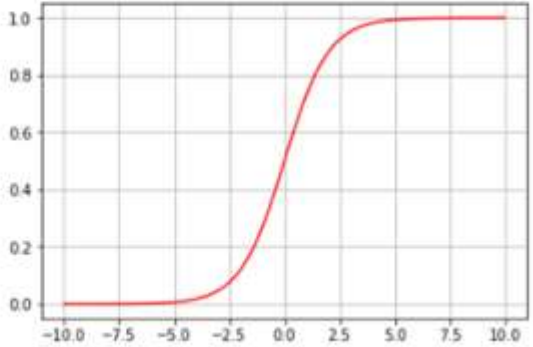
Sigmoid Function in Python

```
In [1]: import numpy as np
        import matplotlib.pyplot as plt
        %matplotlib inline

In [2]: z = np.linspace(-10,10,100)

In [3]: sigmoid =1/(1+np.exp(-z))

In [4]: plt.plot(z, sigmoid,'r')
        plt.grid();
```



Logistic Regression

- We are going to use ***z*** to denote this quantity $w^T x + b$
- Then, we have: $\hat{y} = \sigma(\underbrace{w^T x + b}_{z})$
- If ***z*** is **large**: $\boxed{\sigma(z) = \frac{1}{1+0} \approx 1}$
- If ***z*** is **large negative number**: $\boxed{\sigma(z) = \frac{1}{1+\infty} \approx 0}$
- When we implement logistic regression, our job is to try to **learn parameters w** and **b**, so that $\hat{y}$ becomes a good estimate of the **chance of *y* being equal to 1**.

Logistic Regression Cost Function

FSB

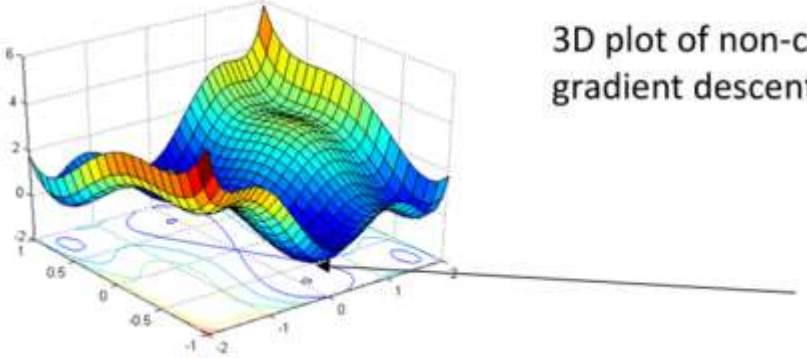
Logistic Regression Cost Function

- To train parameters W and b of your logistic regression model you need to define a cost function.
- Given a training set of m training examples, we want to find parameters W and b so that $\hat{y}$ is as close to y (ground truth).
- We will use (I) superscript to index different training examples.
- We use loss (error) function (L) to measure how well out algorithm is doing. Loss function is applied only to single training sample.

Logistic Regression Cost Function

- Squared Error $L(\hat{y}, y) = \frac{1}{2}(\hat{y}, y)^2$

  - We don't use this one because it makes gradient descent not work well so in logistic regression optimization problem becomes non-convex.



3D plot of non-convex gradient descent

Global minimum

Logistic Regression Cost Function

- We will use following loss function: $L(\hat{y}, y) = -(ylog\hat{y} + (1-y)log(1-\hat{y}))$
  - It will give us convex optimization problem and it is much easier to otimize.
  - Y and ŷ are sigmoid function so they can never bi bigger than 1 or less then 0
  - If y = 1:
    - L(ŷ, y) = - log ŷ
    - Want log ŷ large, want ŷ large (as close to 1 as possible)
  - If y = 0:
    - L(ŷ, y) = - log (1 - ŷ)
    - Want log (1 - ŷ) large, want ŷ small (as close to 0 as possible)

Logistic Regression Cost Function

- Cost function (how well are you doing on the entire training set):
  - J(W,b) = $\frac{1}{m}\sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m} \hat{y}^{(i)} log\hat{y^{(i)}} + (1 - y^{(i)}) log(1 - \hat{y}^{(i)})$
  - J is average of sum of loss functions of all parameters.
  - Cost function is cost of all parameters.
  - We will try to find W and b that minimize the overall cost function J.

GRADIENT DESCENT

FSB

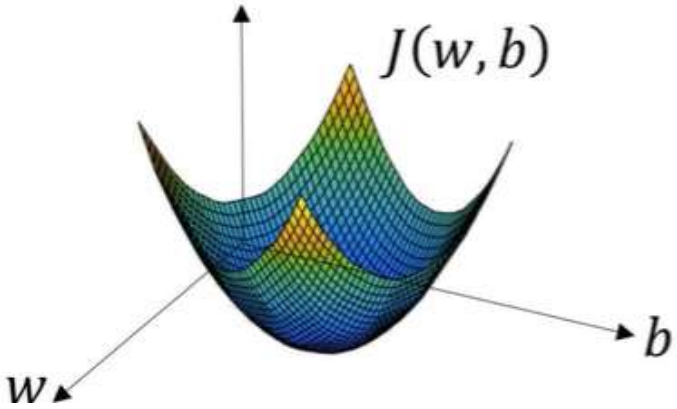
Gradient Descent

- Gradient Descent is a process of trying to minimize the error function.
- Gradient descent algorithm is been used to train and also learn the parameters W on your training set.
- In order to learn the set of parameters W and B it seems natural that we want to find W and B that make the cost function J (W, b) as small.

Recap: $\hat{y} = \sigma(w^T x + b),\ \sigma(z) = \frac{1}{1+e^{-z}}$

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})$$

Want to find w, b that minimize $J(w,b)$

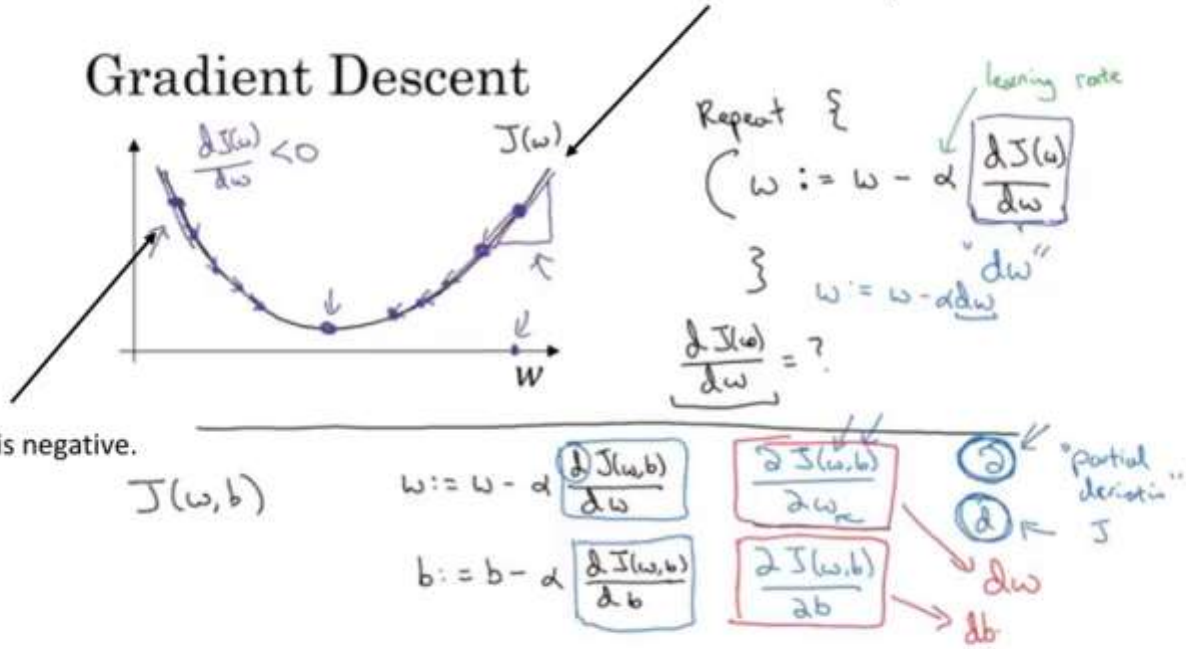


FSB

How does Gradient Descent Work

- Gradient Descent starts at an initial point and began taking steps in the steepest downhill direction.
- After a single step, you might end up there because it's trying to take a step downhill in the direction of steepest descent or quickly down low as possible.
- So after one and two iterations of gradient descent, you might end up with the third iteration and so on, until eventually, hopefully you converge to this global optimum or get to something close to the global optimum.

- When derivative is positive.



- When derivative is negative.

- The learning rate controls how big a step we take on each iteration of gradient descent
- Derivative is basically the update of the change you want to make to the parameters W
- Note : in our code ' dw ' will be used to represent this derivative term.

- Derivative is the slope of a function at the point.

Gradient Descent

These equations are equivalent.

$$w = w - \alpha * \frac{\mathrm{d}J(w,b)}{\mathrm{d}w}$$

$$b = b - \alpha * \frac{\mathrm{d}J(w,b)}{\mathrm{d}b}$$

$$w = w - \alpha * \frac{\partial J(w,b)}{\partial w}$$

$$b = b - \alpha * \frac{\partial J(w,b)}{\partial b}$$

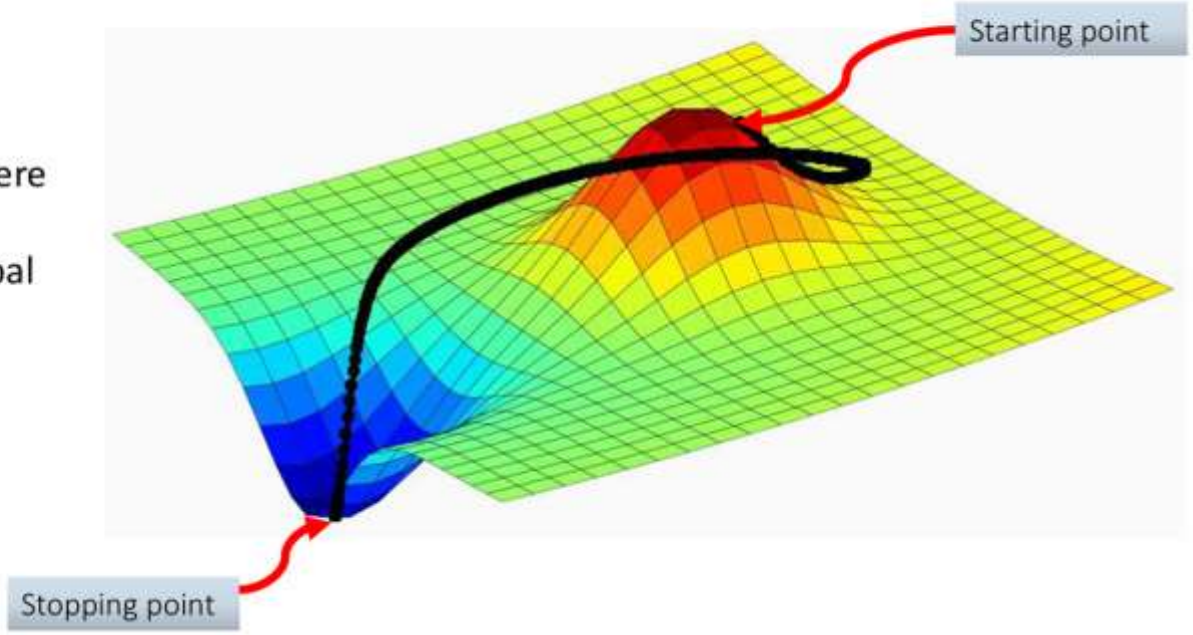
Partial Derivatives - ∂

RULE : If ' J ' is a function of two variables, instead of using the lower case 'd', we use the partial derivative symbol.

Gradient Descent

Visualization in 3D

We can see a picture of the gradient descent algorithm in action, in a 3D visualization, were it is taking baby steps in order and hopefully to reach it's global optimum.



Gradient Descent

- By using the function gradient descent we have a **general** direction in which to go to reduce the error
- **Convex functions** have a **single global optimum**
- **Non-Convex functions** have **different local optima.**
- **Optimization algorithm** tries to alter model parameters " W " and get a smaller error
- Lots of calculation is based on Linear Algebra
- That means we can write lots of things in a matrix notation
- This allows us to **parallelize,** it quite a lot on a GPU

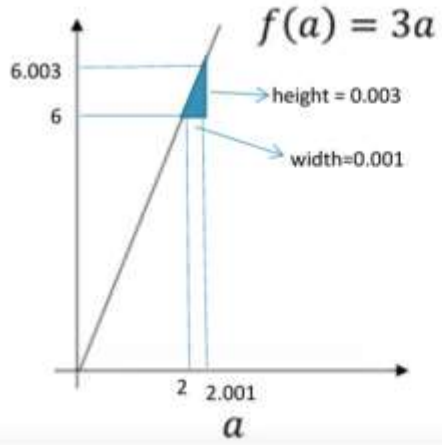
DERIVATIVES

FSB

Derivatives

- We should open up the box and peer a little bit further into the details of calculus but really all you need is an intuitive understanding of this in order to build and successfully apply Deep Learning algorithms.
- The term derivative basically means slope of a function.
- Here the function **f(a) = 3a** is plotted so it's just a straight line to gain intuition about derivatives.
- Slope is a friendlier way to describe the concept of derivative, so **whenever you hear derivative just think slope of the function** and more formally the **slope is defined as the height divided by the width** of this little triangle that we have in blue.

Intuition about derivatives



$$slope = \frac{height}{width}$$

$$slope = \frac{0.003}{0.001} = 3$$

$$a = 2, f(a) = 6$$
$$a = 2.001, f(a) = 6.003$$

slope (derivative) of f(a) for a = 2 and a = 2.001 **is 3**

$$a = 5, f(a) = 15$$
$$a = 5.001, f(a) = 15.003$$

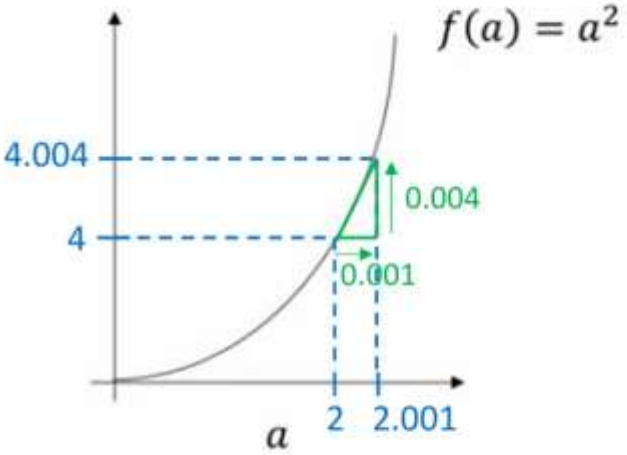
slope (derivative) of f(a) for a = 5 and a = 5.001 **is also 3**

$$\frac{\mathrm{d}f(a)}{\mathrm{d}a} = 3 = \frac{\mathrm{d}}{\mathrm{d}a}f(a)$$

Whenever ***a*** is nudged by 0.1 or 0.001 or 0.00...0001, **slope will always be 3 for function *f(a) = 3a***

More Derivatives Examples

Intuition about Derivatives



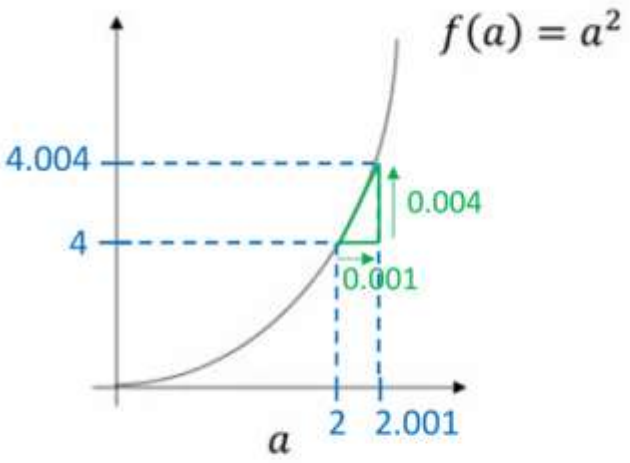
$a = 2$ $\quad f(a) = 4$

$a = 2.001$ $\quad f(a) \approx 4.004$

- We have plotted the function **f(a) = a²**, taking a look at the point **a=2**, **a²** or **f(a) = 4**.
- If we move a slightly to the right, **a=2.001**, **f(a)** which is **a²** is going to be approximately **4.004**.
- What this means is when **a=2**?
- What we're saying is that if **a=2**, then **f(a) = 4** (here the x and y axis are not drawn to scale).
- If we move **a** to **2.001**, then **f(a)** becomes roughly **4.004**.
- So if we draw a little triangle, this means is that if we move it to the right by **0.001**, **f(a)** goes up four times as much by **0.004**.

More Derivatives Examples

Intuition about Derivatives



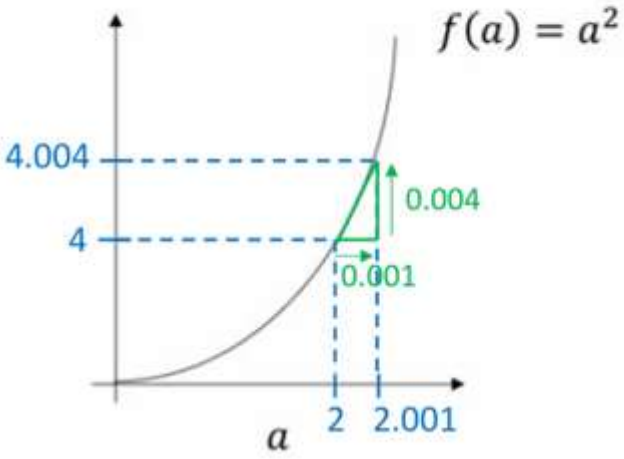
$a = 2$ $\quad f(a) = 4$

$a = 2.001$ $\quad f(a) \approx 4.004$

- In the language of calculus, we say that a **slope** that is the **derivative** of **f(a)** at **a=2** is **4**
- Calculus notation: $\boxed{\frac{\mathrm{d}}{\mathrm{d}a} f(a) = 4, \text{ when } a = 2}$
- We say that **d/da** of **f(a) = 4** when **a=2.**
- Now one thing about this function **f(a) = a²** is that **the slope is different for different values of a**.

More Derivatives Examples

Intuition about Derivatives



- If **a=5**, then **a²=25**, so that's **f(a).**
- If we move **a** to the right again, it's tiny little change to **a**, now **a=5.001**, then **f(a)** will be approximately **25.010**.
- So what we see is that by moving **a** up by 0.001, **f(a)** goes up ten times as much 0.010.
- We have that: $\boxed{\frac{\mathrm{d}}{\mathrm{d}a} f(a) = 10, \text{ when } a = 5}$
- When **a=5** because **f(a)** goes up ten times as much as **a** does when we make a tiny little nudge to **a**.

COMPUTATION GRAPH

FSB

Computation Graph

- The computations of a neural network are organized in terms of
  - a **forward pass** or a **forward propagation step**, in which we compute the output of the neural network,
  - followed by a **backward pass** or **back propagation step**, which we use to compute gradients or compute derivatives.
- The **computation graph** explains why it is organized this way.
- In order to illustrate the computation graph, let's use a simpler example than logistic regression or a full blown neural network.

Computation Graph

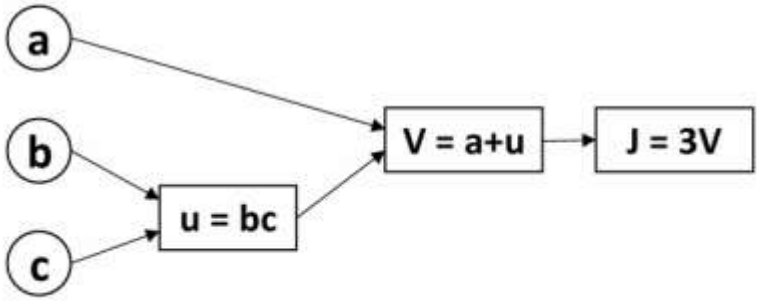
- Let's say that we're trying to compute a **function J**, which is a function of **three variables a, b,** and **c** and let's say that **function J** is **3(a+bc)**.
- Computing this function actually has three distinct steps:
  1. Compute **bc** and store it in the variable **u,** so **u=bc**
  2. Compute **V=a u,**
  3. Output **J** is **3V**.

J(a,b,c)=3(a+bc)

u=bc

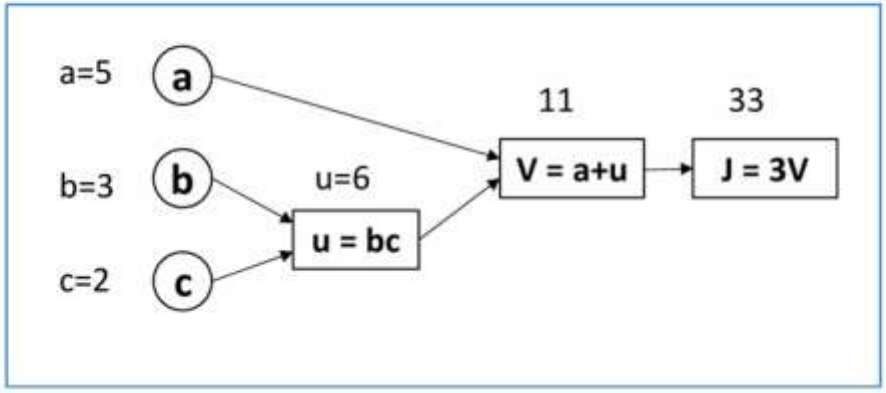
V=a+u

J=3V



Derivatives with a Computation Graph

- We have learned how we can use a **computation graph to compute the function J**.
- How we can use a computation graph to figure out **derivative calculations of the function J**.
- Using a computation graph we want to compute the **derivative of J** with **respect to v**.



Derivatives with a Computation Graph

- So what is that?
- If we were take this value of **v** and change it a little bit, how would the value of **J** change?

$$J = 3v$$

$$v = 11 \mapsto 11.001$$

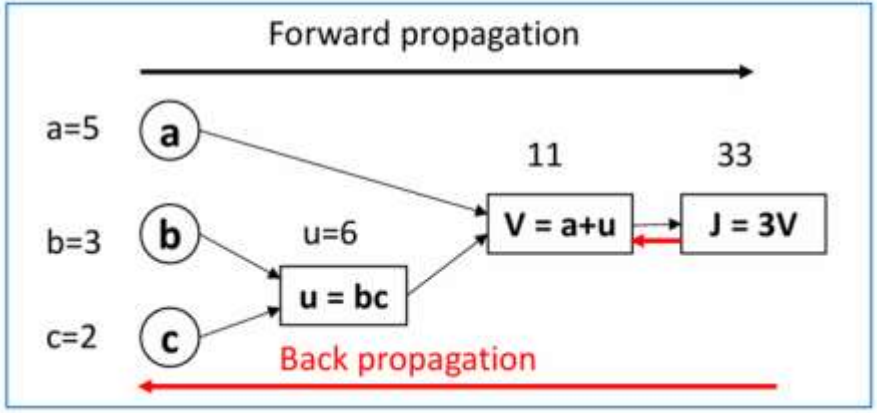
$$J = 33 \mapsto 33.003$$

$$\frac{dJ}{dv} = 3$$

- Analogous example

$$f(a) = 3a \quad \frac{df}{da} = 3$$

- Calculating dJ/dv is a one step of **back propagation.**



Derivatives with a Computation Graph

$$a = 5 \mapsto 5.001$$
$$v = 11 \mapsto 11.001$$
$$J = 13 \mapsto 13.003$$
$$\frac{dJ}{da} = \frac{dJ}{dv}\frac{dv}{da}$$
$$\frac{dJ}{da} = 3$$

- One way to break this down is to say that if you change ***a*** then that would change ***v*** and through changing ***v*** that would change ***J***.
- By increasing **a**, how much is **v** increased? This is determined by **dv/da**.
- Then, the change in **v** will cause the value of **J** to also increase. This is called a **chain rule** in calculus:

$$\frac{dJ}{da} = \frac{dJ}{dv}\frac{dv}{da}$$

FSI

Derivatives with a Computation Graph

- In this course we will use the following notation:

$$\frac{\mathrm{d}FinalOutputVar}{\mathrm{d}Var} -->" dVar "$$

- So, in our code will use:

$$\frac{\mathrm{d}J}{\mathrm{d}v} -->" dv "$$

$$\frac{\mathrm{d}J}{\mathrm{d}a} -->" da "$$

FSB

Derivatives with a Computation Graph

- What is the derivative dJ/du?

$$\frac{dJ}{du} = ?$$

$$u = 6 \mapsto 6.001$$

$$v = 11 \mapsto 11.001$$

$$J = 33 \mapsto 33.003$$

$$\frac{dJ}{du} = \frac{dJ}{dv}\frac{dv}{da} = 3 \cdot 1$$

- A question when working with derivatives is: How much do we need to **change b** in order to **maximize/minimize J**? What is the slope when we change b?
- When we **change b**, we will **change u**, and **u will affect J**.

$$\frac{dJ}{db} = ?$$

$$b = 3 \mapsto 3.001$$

$$u = 6 \mapsto 6.002$$

$$J = 33 \mapsto 33.006$$

$$\frac{du}{db} = 2$$

$$\frac{dJ}{db} = \frac{dJ}{du}\frac{du}{db} = 3 \cdot 2 = 6$$

Derivatives with a Computation Graph

- What is the derivative dJ/dc?

$$\frac{\mathrm{d}J}{\mathrm{d}c} = ?$$

$$c = 2 \mapsto 2.001$$

$$u = 6 \mapsto 6.003$$

$$J = 33 \mapsto 33.009$$

$$\frac{\mathrm{d}u}{\mathrm{d}c} = 3$$

$$\frac{\mathrm{d}J}{\mathrm{d}c} = \frac{\mathrm{d}J}{\mathrm{d}u}\frac{\mathrm{d}u}{\mathrm{d}c} = 3 \cdot 3 = 9$$

FSB

LOGISTIC REGRESSION DERIVATIVES

FSB

Logistic Regression Recap

- **Z** - Logistic Regression Formula

$$z = w^T x + b$$

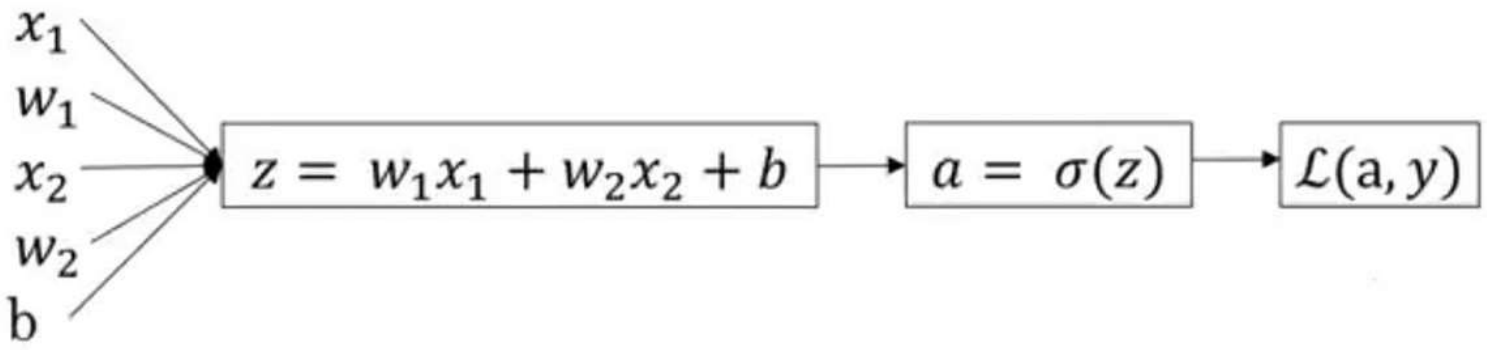
- **Y** predictions

$$\hat{y} = a = \sigma(z)$$

- Loss Function where **a** is the output of logistic regression and **y** is the ground truth label

$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

Logistic regression derivatives



FSB

Logistic Regression Derivatives

- In logistic regression all what we want to do is to modify the parameters **W** and **B** in order to reduce this loss.
- First we have to compute the loss on a single training using the forward propagation steps.
- Then we go backwards to compute the derivatives.

Logistic Regression Derivatives

How do we compute the derivatives.

- Compute the derivative of $\frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}a}$ and in code we would denote this as $da = \frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}a}$

  Note when writing code I assigned $\frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}a}$ to "**da**"

$$da = -\frac{y}{a} + \frac{1-y}{1-a}$$

- Having computed "**da**", we can go backwards to compute "**dz**"

$$dz = \frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}z}$$

$$dz = \frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}a} * \frac{\mathrm{d}a}{\mathrm{d}z}$$

$$dz = a - y$$

Since we know that :

$$\frac{\mathrm{d}a}{\mathrm{d}z} = a(1-a)$$

$$\frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

Logistic Regression Derivatives

- The final step in back propagation is to go back to compute how much we need to change **w** and **b**

$$dw1 = \frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}w1} = x1 * dz \qquad dw2 = \frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}w2} = x2 * dz \qquad db = \frac{\mathrm{d}\mathcal{L}(a,y)}{\mathrm{d}b} = dz$$

- so if you want to do gradient descent with respect to just this one example, we would do the following by performing this updates.

$$w1 = w1 - \alpha * dw1 \qquad w2 = w2 - \alpha * dw2 \qquad b = b - \alpha * db$$

GRADIENT DESCENT ON m TRAINING EXAMPLES

FSB

Gradient Descent on m Training Examples

- The derivative respect to w1 of the overall cost function is also going to be the average of derivatives respect to w1 of the individual loss terms.

$$\frac{\mathrm{d}}{\mathrm{d}w_1} J(w, b) = \frac{1}{m} \sum_{i=1}^{m} \frac{\mathrm{d}}{\mathrm{d}w_1} L(a^{(i)}, y^{(i)})$$

- Previously we have already shown how to compute this term for a single training sample:

$$dw_1^{(i)} = \frac{\mathrm{d}}{\mathrm{d}w_1} L(a^{(i)}, y^{(i)})$$

- We need to compute this derivative on the previous training example and average them and we will get the overall gradient that we can use to implement gradient descent.

Gradient Descent on m Training Examples

- Let's wrap this up into a concrete algorithm and what you should implement to get logistic regression with gradient descent working. We do this calculation assuming that we have two features so that n = 2, otherwise we do this for dw1, dw2, db ...

$$J = 0; dw_1 = 0; dw_2 = 0; db = 0$$

For i = 1 to m

$$Z^{(i)} = w^T x^{(i)} + b$$
$$a^{(i)} = \sigma(Z^{(i)})$$
$$J+ = -[y^{(i)} log a^{(i)} + (1 - y^{(i)}) log(1 - a^{(i)}))]$$
$$dz^{(i)} = a^{(i)} - y^{(i)}$$
$$dw_1+ = x_1^{(i)} dz^{(i)}$$
$$dw_2+ = x_2^{(i)} dz^{(i)}$$
$$db+ = dz^{(i)}$$

n

Gradient Descent on m Training Examples

- After we leave for loop we need to divede all by m because we are computing averages:

$$J/ = m;$$
$$dw_1/ = m;$$
$$dw_2/ = m;$$
$$db/ = m;$$

- With all of these calculations, we computed derivative of the cost function J with respect to parameters w1, w2, and b.

Gradient Descent on m Training Examples

- We're using dw1 and dw2 and db as accumulators.

$$dw_1 = \frac{\mathrm{dJ}}{\mathrm{d}w_1}; dw_2 = \frac{\mathrm{dJ}}{\mathrm{d}w_2}; db = \frac{\mathrm{dJ}}{\mathrm{d}b}$$

- We use them to sum the entire training set.
- After all this calculations to implement one step of gradient descent we implement

$$w_1 := w_1 - \alpha dw_1$$
$$w_2 := w_2 - \alpha dw_2$$
$$b := b - \alpha db$$

Gradient Descent on m Training Examples

- It turns out there are two weaknesses with calculation as we've implemented in here.
- To implement logistic regression this way we need to write two for loops (loop over m training samples and loop over all the features).
- When we are implementing deep learning algorithms we find that explicit for loops make our algorithms run less efficiency so if we want to scale to bigger data sets we must avoid them.
- For that we use vectorization.

VECTORIZATION

FSB

Vectorization

- **Vectorization** is a style of computer programming where operations are **applied to whole arrays instead of individual elements**
- Basically the art of **getting rid of explicit for loops** in your code.
- Deep learning algorithms tend to shine when training on large datasets, so it's important that your code runs quickly. Otherwise, your code might take a long time to get your result.
- Vectorization can significantly **speed up your code**.
- The rule of thumb to remember is whenever **possible avoid using explicit for loops in your code.**
- In the next slides, I would be doing a comparison in time using a vectorized and non-vectorized implementation to compute the dot products of two arrays.

Vectorization

Vectorized Implementation.

In [1]:
```
import numpy as np
```

In [2]:
```
a = np.random.rand(1000000)
b = np.random.rand(1000000)
```

In [3]:
```
# Vectorized Implementation
```

In [4]:
```
%%timeit
c = np.dot(a, b)
```

```
597 µs ± 72.6 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

Vectorized Implementation : 597 nanoseconds

FSB

Vectorization

Non-Vectorized Implementation.

In [1]:
```
import numpy as np
```

In [2]:
```
a = np.random.rand(1000000)
b = np.random.rand(1000000)
```

In [3]:
```
# Non-Vectorized Implementation
```

In [4]:
```
%%timeit
c = 0
for i in range(1000000):
    c += a[i] * b[i]
```

```
405 ms ± 67 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Vectorized Implementation : 405 milliseconds

FSB

Vectorization

- Our results clarifies that having a vectorized implementation in your code, does actually make your code run faster.
- The **vectorized Implementation** took **597 nanoseconds** (Better)
- While, the **non-vectorized implementation** took **407 milliseconds**.
- NumPy is the fundamental package for scientific computing in Python.

More Vectorization Examples

- When we are programming neural networks or logistic regression we should avoid explicit for loops.
- It isn't always possible but when we can we should use built-in functions or find some other ways to compute.
- Some examples:
  - Non vectorized:

$$u = AV$$
$$u_i = \sum_j A_{ij} V_j$$

$$u = np.zeros((n,1))$$
$$for\ i\ \ldots$$
$$\quad for\ i\ \ldots$$
$$\quad\quad u[i] += A[i][j] * V[j]$$

Vectorized:

U=np.dot(A,V)

More Vectorization Examples

- Let's apply the exponential operation on every element of matrix/vector.
- Non vectorized:

$$
\begin{aligned}
&v = [v_1, v_2, ..., v_m] \\
&u = [ev_1, ev_2, ..., ev_m] \\
&u = np.zeros((n, 1)) \\
&for\ i\ in\ range(n): \\
&\quad u[i] = math.exp(v[i])
\end{aligned}
$$

- Python library numpy has many functions that help us with vectorization.
- Vectorized:

```
Import numpy as np
U = np.exp(v)
```

More Vectorization Examples

- Some other numpy vectorization functions:
  - Np.log
  - Np.abs
  - Np.maximum
  - V**2
  - 1/v

FSB

More Vectorization Examples

- Let's apply this on our logistic regression gradient descent to remove our for loops.

$$
\begin{aligned}
&J = 0, dw = np.zeros((nx, 1)), db = 0 \\
&for\ i = 1\ to\ m: \\
&\quad z^{(i)} = w^T x^{(i)} + b \\
&\quad a^{(i)} = \alpha(z^{(i)}) \\
&\quad J+ = -[y^{(i)} log\hat{y}^{(i)} + (1 - y^{i}) log(1 - \hat{y}^{(i)})] \\
&\quad dz^{(i)} = a^{(i)}(1 - a^{(i)}) \\
&\quad dw+ = x^{(i)} dz^{(i)} \\
&\quad db+ = dz^{(i)} \\
&J = J/m, dw/ = m, db = db/m
\end{aligned}
$$

- We still have one for loop but code is running much faster.

VECTORIZING LOGISTIC REGRESSION

FSB

Vectorizing Logistic Regression

- We are going to learn how to vectorize our implementation of logistic regression
- We want to process an entire training set that has implemented a single iteration of gradient descent with respect to the entire training set without using even a single explicit for loop.
- First we need to compute the forward propagation step of logistic regression, by computing this below :

$$z^{(1)} = w^T x^{(1)} + b \qquad z^{(2)} = w^T x^{(2)} + b \qquad z^{(3)} = w^T x^{(3)} + b$$

$$a^{(1)} = \sigma(z^{(1)}) \qquad a^{(2)} = \sigma(z^{(2)}) \qquad a^{(3)} = \sigma(z^{(3)})$$

- First we need to compute Z, then compute also the activation function.
- For M training examples, you have to carry out the forward propagation step that is to compute these predictions on all M training example.

Vectorizing Logistic Regression

- First we stack together in different columns our training inputs and assign them to the variable $X = [x^{(1)}, x^{(2)}, ..., x^{(n)}]$ which is an Nx by M dimensional matrix.
- w^T (w transpose) would be a row vector.
- b is a 1 by M matrix or an M dimensional row vector.
- Next we can compute our Z as a 1 by M matrix

$$Z = [z^1, z^2, ..., z^n] = w^T * X + [b, b, ..., b] = [w^T x^1 + b, w^T x^2 + b, ..., w^T x^n + b]$$

- In python :

$$Z = np.dot(w.T, X) + b$$

- First we stack together in different columns our training inputs and assign them to the variable $X = [x^{(1)}, x^{(2)}, ..., x^{(n)}]$ which is an Nx by M dimensional matrix.
- w^T (w transpose) would be a row vector.
- b is a 1 by M matrix or an M dimensional row vector.
- Next we can compute our Z as a 1 by M matrix

$$Z = [z^1, z^2, ..., z^n] = w^T * X + [b, b, ..., b] = [w^T x^1 + b, w^T x^2 + b, ..., w^T x^n + b]$$

- In python :

$$Z = np.dot(w.T, X) + b$$

Vectorizing Logistic Regression

- Next we want to find a way to compute A all at the same time.

$$A = [a^{(1)}, a^{(2)}, ..., a^{(n)}] = \sigma(Z)$$

- Formula :

$$\sigma = \frac{1}{1 + e^{-Z}}$$

- In python :

```
In [1]: import math

        def sigmoid(x):
            return 1 / (1 + math.exp(-x))

In [2]: sigmoid(0.458)

Out[2]: 0.6125396134409151
```

FSB

Vectorizing Logistic Regression's Gradient Computation

- We are going to learn how to compute all samples at a same time.
- We need to define new variable dZ which will contain all dz values stacked horizontally in 1 by m matrix.

$$dZ = [dz^{(1)}, dz^{(2)}, ..., dz^{(m)}]$$
$$dZ = A - Y = [a^{(1)} - y^{(1)}, a^{(2)} - y^{(2)}, ...]$$

- In the previous example, we have gotten rid of one for loop but the second one (for loop over m training samples) still remained.

Vectorizing Logistic Regression's Gradient Computation

- Now we will vectorize all. Starting with vectorization of db and dw.

$$db = \frac{1}{m}\sum_{i=1}^{m} dz^{(i)} \qquad dw = \frac{1}{m}XdZ^T = \frac{1}{m}[x^{(1)}dz^{(1)} + ... + x^{(m)}dz^{(m)}]$$

- In python:

$$db = \frac{1}{m}np.sum(dZ) \qquad dw = \frac{1}{m}XdZ^T$$

Vectorizing Logistic Regression's Gradient Computation

- Let's pull it all together to actually implement logistic regression.

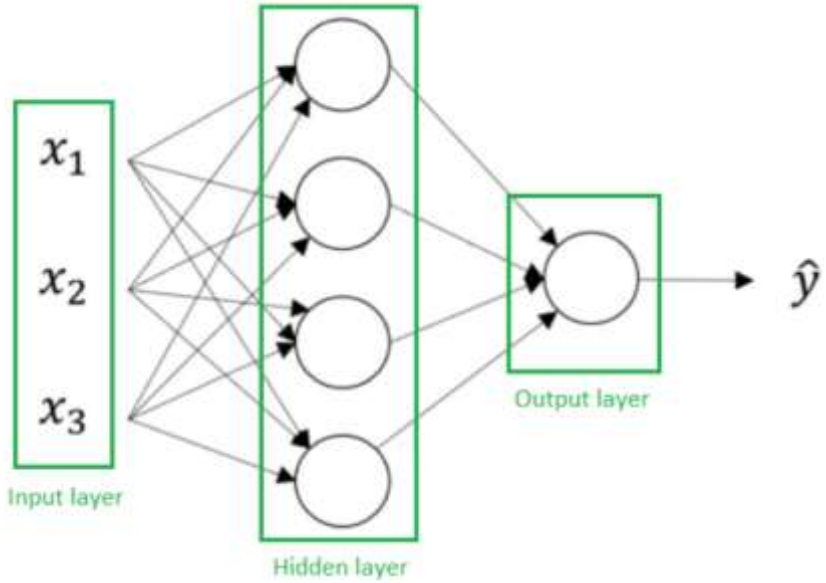
$$Z = np.dot(w.T, X) + b$$
$$A = \alpha(Z)$$
$$dZ = A - Y$$
$$dw = \frac{1}{m} X\, dZ^T$$
$$db = \frac{1}{m} np.sum(dZ)$$
$$w := w - \alpha dw$$
$$b := b - \alpha db$$

Vectorizing Logistic Regression's Gradient Computation

- With this, we implemented single iteration of logistic regression.
- If we want to implement logistic regression over a number of iteration we need for loop.
- We can't get rid of that for loop.

FSB

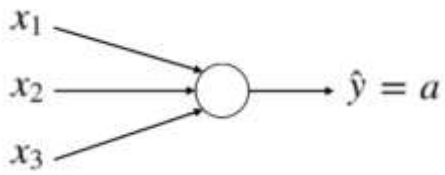
Neural Networks Representation



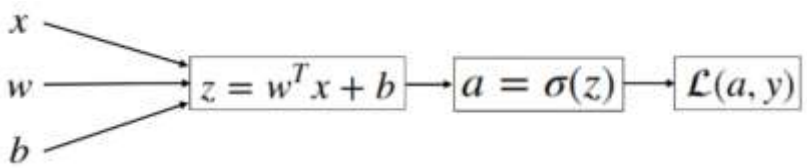
FSB

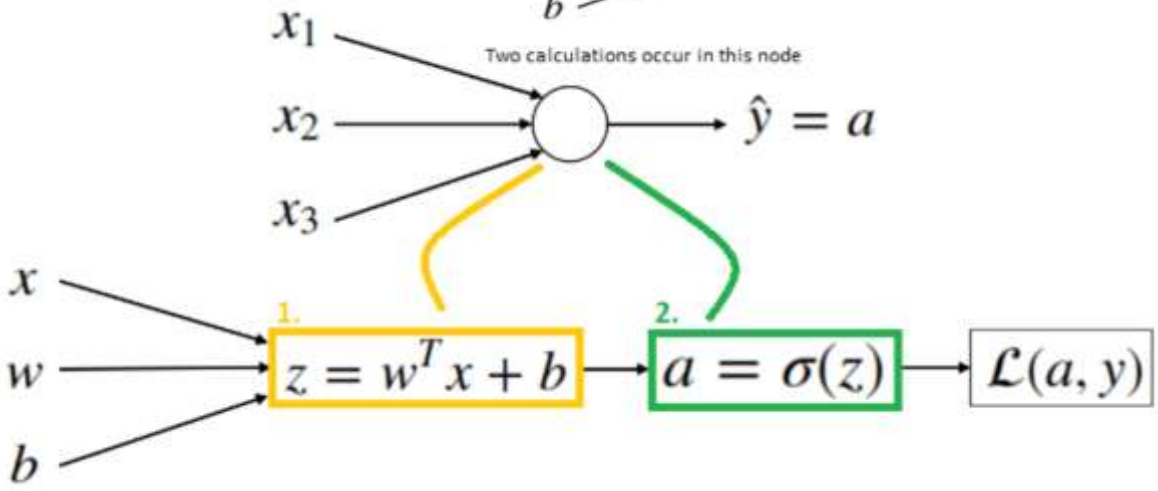
Quick Overview

- Logistic Regression



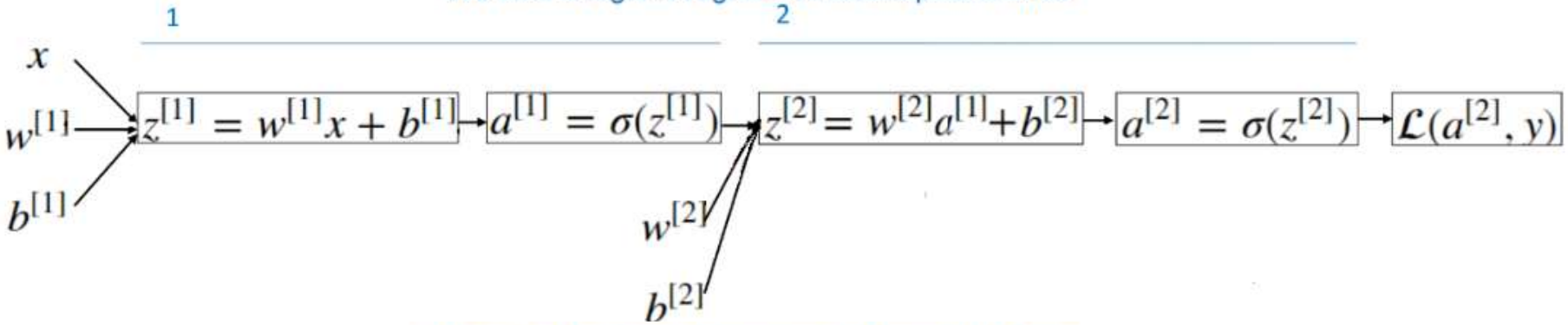
- Computation graph



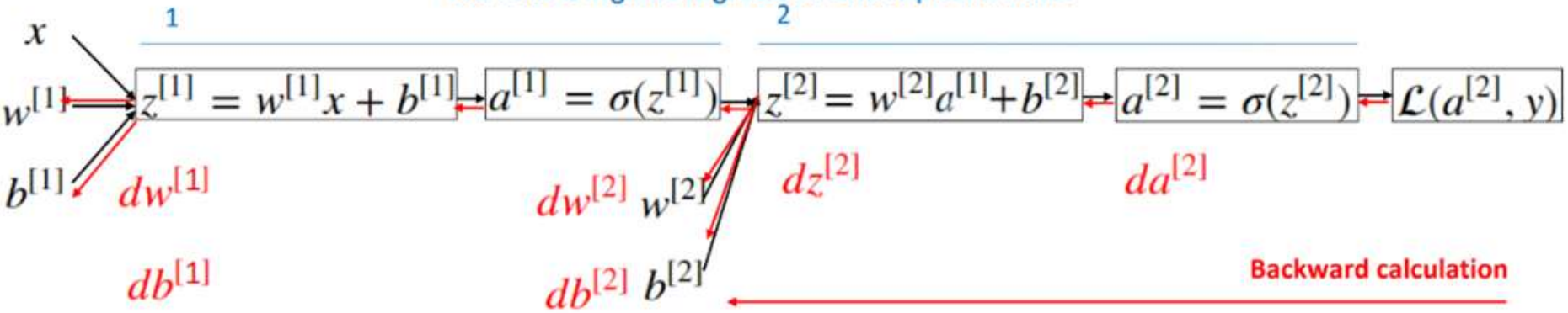


Forward –Backward Propagation

We take a logistic regression and repeat it twice

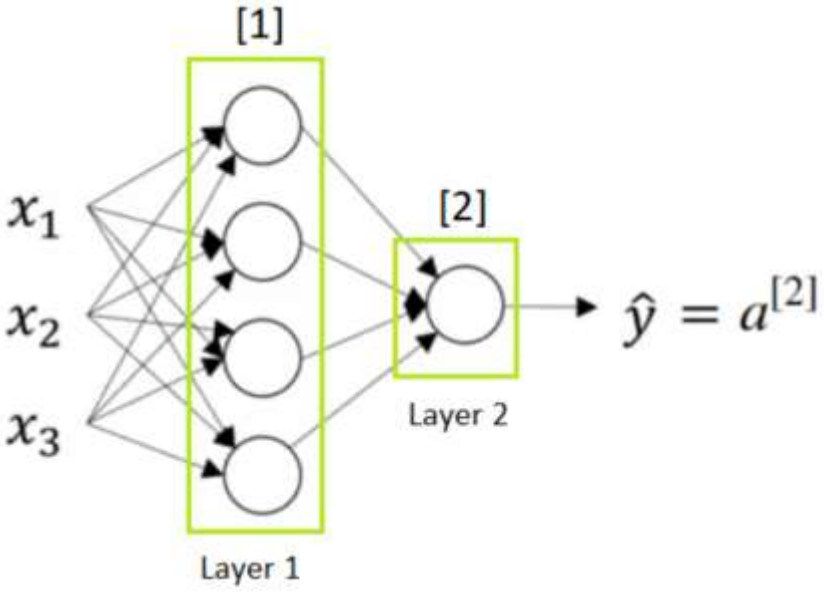


We take a logistic regression and repeat it twice



Computing a Neural Network output

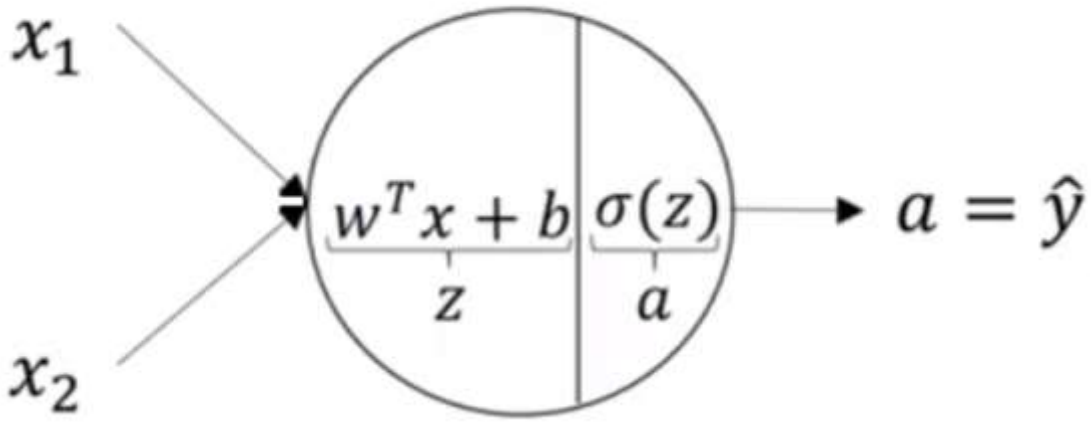
- A neural network is created by stacking together several node units.
- One stack of nodes we will call a layer.
- A superscript [1] refers to the quantities associated with the first stack of nodes, called Layer 1.
- The same is with a superscript [2] which refers to the second layer.
- Remember also that x(i) refers to an individual training example.



FSB

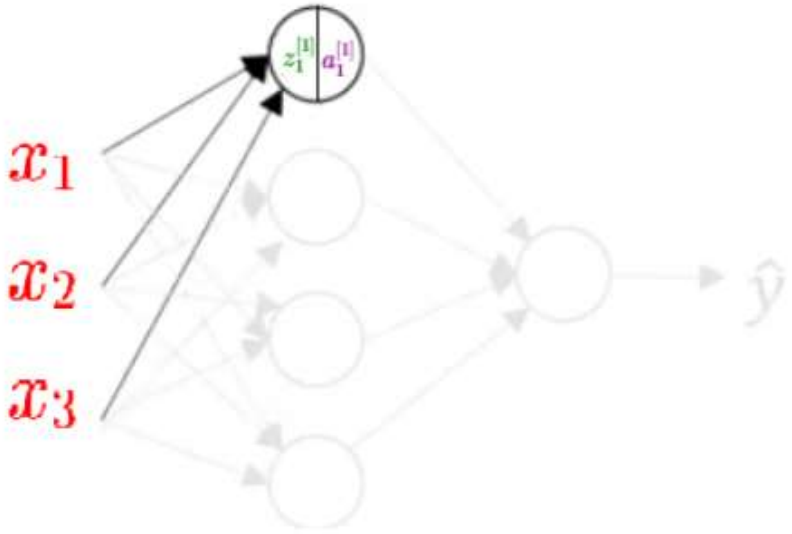
Computing a Neural Network output

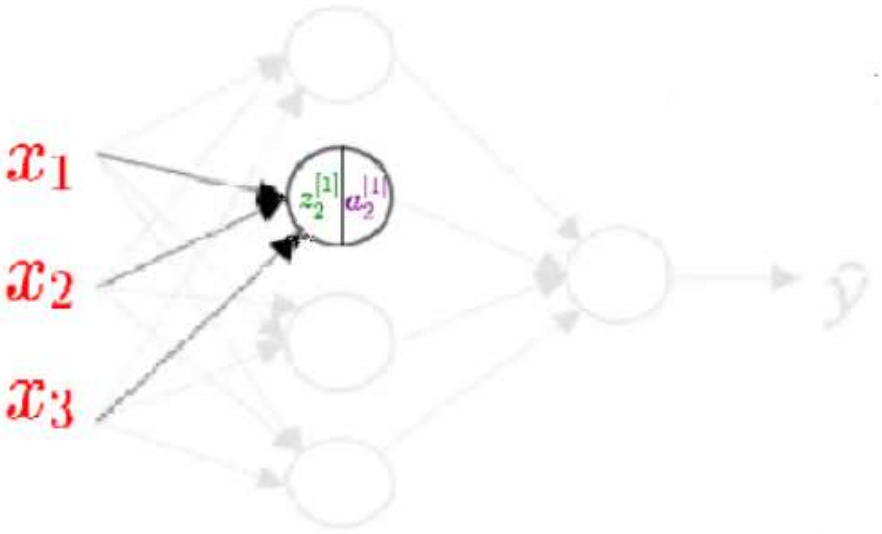
- Computing an output of a Neural Network is like computing an output in Logistic Regression, but repeating it multiple times.



FSB

Computing a Neural Network output





$$z_1^{[1]} = w_1^{[1]^T} x + b_1^{[1]}$$

$$a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = w_2^{[1]^T} x + b_2^{[1]}$$

$$a_2^{[1]} = \sigma(z_2^{[1]})$$

Computing a Neural Network output

Calculations for the third and fourth node look the same. Now, we will put all these equations together:

$$z_1^{[1]} = {w_1^{[1]}}^T x + b_1^{[1]} \qquad a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = {w_2^{[1]}}^T x + b_2^{[1]} \qquad a_2^{[1]} = \sigma(z_2^{[1]})$$

$$z_3^{[1]} = {w_3^{[1]}}^T x + b_3^{[1]} \qquad a_3^{[1]} = \sigma(z_3^{[1]})$$

$$z_4^{[1]} = {w_4^{[1]}}^T x + b_4^{[1]} \qquad a_4^{[1]} = \sigma(z_4^{[1]})$$

Calculating all these equations with for loop is highly inefficient so we will need to vectorize this.

$$\begin{bmatrix} - & {w_1^{[1]}}^T & - \\ - & {w_2^{[1]}}^T & - \\ - & {w_3^{[1]}}^T & - \\ - & {w_4^{[1]}}^T & - \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix} = \begin{bmatrix} {w_1^{[1]}}^T x + b_1^{[1]} \\ {w_2^{[1]}}^T x + b_2^{[1]} \\ {w_3^{[1]}}^T x + b_3^{[1]} \\ {w_4^{[1]}}^T x + b_4^{[1]} \end{bmatrix} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix}$$

FSB

Computing a Neural Network output

So we can define these matrices:

$$W^{[1]} = \begin{bmatrix} - & w_1^{[1]T} & - \\ - & w_2^{[1]T} & - \\ - & w_3^{[1]T} & - \\ - & w_4^{[1]T} & - \end{bmatrix} \qquad b^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

$$z^{[1]} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} \qquad a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix}$$

FSB

Computing a Neural Network output

- To compute the output of a Neural Network we need the following four equations. For the first layer of a Neural network we need these equations:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$dimensions\ \ are: (4,1) = (4,3)(3,1) + (4,1)$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$dimensions\ \ are: (4,1) = (4,1)$$

and for the second:

$$z^{[2]} = W^{[2]}x + b^{[2]}$$

$$dimensions\ \ are: (1,1) = (1,4)(4,1) + (1,1)$$

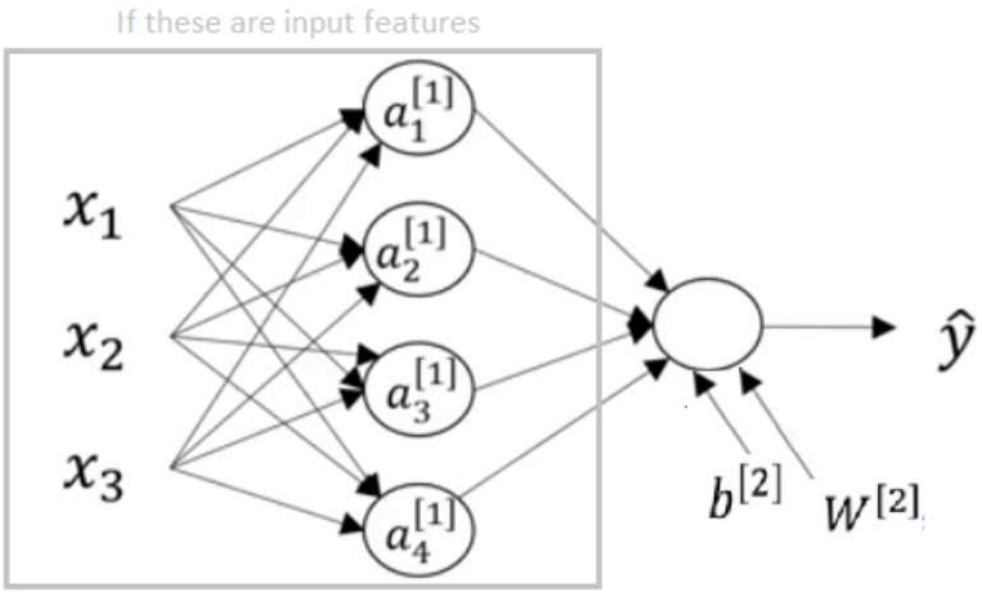
$$a^{[2]} = \sigma(z^{[2]})$$

$$dimensions\ \ are: (1,1) = (1,1)$$

FSB

Computing a Neural Network output

- Calculating the output of the Neural Network is like calculating a Logistic Regression with parameters $W^{[2]}$ as w^T and $b^{[2]}$ as b.



FSB